

신경망

한경수
성결대학교 컴퓨터공학과

* 본 자료는 머신러닝 수업 수강 용도로만 사용이 가능함.
* 본 자료는 대부분 수업의 교재인 아래 도서를 기반으로 작성되었음.
해즈온 머신러닝(3판), 오렐리안 제롬 지음, 박해선 옮김, 한빛미디어, 2023

학습 목표



- ◆ 공학 기술 및 도구 활용 역량
 - 신경망의 학습 원리를 이해하고 설명할 수 있다.
- ◆ 요구 이해 및 설계 역량
 - 머신러닝 라이브러리를 이용하여 신경망 모델을 학습하고 학습 결과를 적용할 수 있다.

Introduction

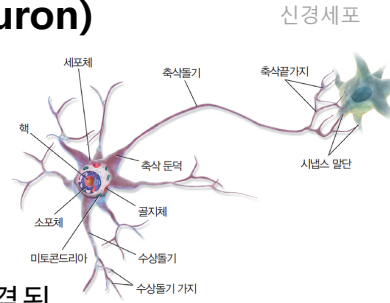


- ◆ 인간처럼 생각하고 행동하는 시스템이 인공지능
- ◆ 학습 기능을 구현하려면 인간의 뇌 동작을 구현하면 되지 않을까?
- ◆ 인간의 뇌 구조로부터 영감을 받아 인공적으로 구현한 신경망: 인공 신경망(artificial neural network; ANN)

신경망 개요

생물학적 뉴런(Neuron)

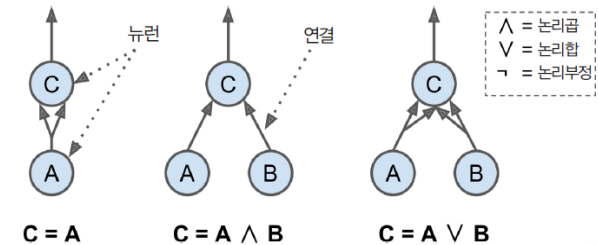
- ◆ 수상돌기(dendrite)
 - 다른 뉴런에게서 정보 수신
- ◆ 축삭돌기(axon)
 - 다른 뉴런에게 정보 발송
 - 끝 부분의 시냅스(synapse)를 통해 다른 뉴런의 수상돌기나 세포체에 연결됨
- ◆ 세포체(cell body)
 - 핵 포함
 - 여러 뉴런에게서(수상돌기) 받은 신호의 합이 뉴런 고유의 임계값을 넘으면(세포체) 다른 뉴런에 신호 전달(축삭돌기)



인공 뉴런

입력 신호 합, 그 결과에 따라 신호 출력

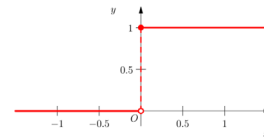
- ◆ 하나 이상의 이진(binary) 입력(On/Off)과 이진 출력 하나를 가짐
- ◆ 입력이 일정 개수만큼 활성화되었을 때 뉴런이 출력을 활성화시킴
 - 예: 입력이 둘 이상 활성화됐을 때 뉴런이 활성화 된다고 가정



TLU

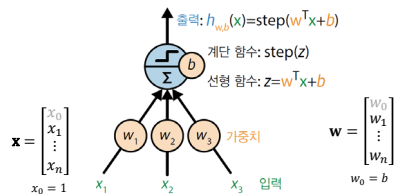
Warren McCulloch and Walter Pitts (1943)

- ◆ threshold logic unit; LTU(linear threshold unit)라고도 불리는 조금 다른 형태의 뉴런
 - 입력과 출력: 숫자 값
 - 각 입력 연결에는 가중치가 연관되어 있음
 - 1)입력 가중 합(선형결합)을 계산한 뒤
 - 2)계단 함수(step function)를 적용하여 결과 출력
- ◆ 간단한 선형 이진 분류 문제에 사용 가능
 - 입력의 선형결합이 임계값을 넘으면 양성 클래스 출력



연결 가중치

활성화 함수(activation function)



벡터 입력(x) → 가중 합 계산(z) → 활성화 함수 적용(step(z))
→ 스칼라 값 출력; 결국 뉴런 하나는 값 하나에 대응됨

$$z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b = \mathbf{w}^T \mathbf{x} + b$$

$$z = w_0 \cdot 1 + w_1 x_1 + w_2 x_2 + \dots + w_n x_n = \mathbf{w}^T \mathbf{x}$$

뉴런마다 서로 다른 임계값 적용 편향(bias)

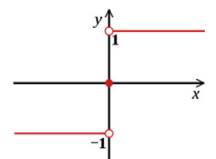
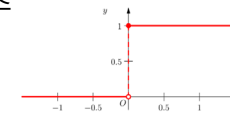
퍼셉트론(Perceptron)

- ◆ 가장 간단한 인공 신경망 구조 중 하나
 - 1957년 Frank Rosenblatt이 제안
- ◆ TLU를 기반으로 함
- ◆ 퍼셉트론에서 일반적으로 사용하는 계단 함수
 - 헤비사이드(Heaviside) 계단 함수

$$\text{heaviside}(z) = \begin{cases} 0 & z < 0 \text{일 때} \\ 1 & z \geq 0 \text{일 때} \end{cases}$$

- 부호 함수(sign function)

$$\text{sgn}(z) = \begin{cases} -1 & z < 0 \text{일 때} \\ 0 & z = 0 \text{일 때} \\ +1 & z > 0 \text{일 때} \end{cases}$$



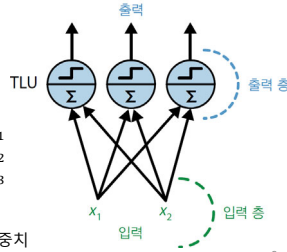
퍼셉트론

뉴런들이 모인 한 층은
결국 하나의 벡터에 대응됨

- 한 층의 TLU들로 구성됨
 - 각 TLU는 모든 입력에 연결됨
 - 한 층에 있는 모든 뉴런이 이전 층의 모든 뉴런과 연결되어 있을 때 **완전 연결 층**(fully connected layer) 또는 **밀집 층**(dense layer)라고 부름
- 입력은
입력 층(input layer)을 구성
- TLU의 층은
출력 층(output layer)

$$\begin{aligned} z_1 &= w_{11}x_1 + w_{21}x_2 + b_1 \\ z_2 &= w_{12}x_1 + w_{22}x_2 + b_2 \\ z_3 &= w_{13}x_1 + w_{23}x_2 + b_3 \end{aligned}$$

w_{12} : 첫번째 입력과 두번째 뉴런의 연결 가중치



한경수

9

퍼셉트론

$$z_j = w_{1j}x_1 + w_{2j}x_2 + \dots + w_{nj}x_n$$

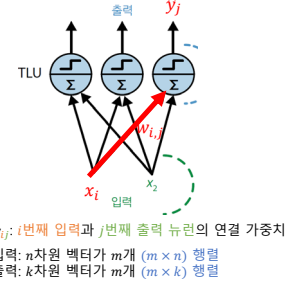
- 완전 연결 층의 출력

- 여러 샘플들에 대한 한 층의 인공 뉴런 출력들을 한꺼번에 효율적으로 계산 가능

$$h_{\mathbf{w}, \mathbf{b}}(\mathbf{X}) = \phi(\mathbf{XW} + \mathbf{b}) \quad (m \times k) \quad \mathbb{R}^n \rightarrow \mathbb{R}^k$$

- $\mathbf{X}$: 입력 특성들의 행렬 ($m \times n$)
 - 각 행은 각 샘플에, 각 열은 각 특성에 해당함
- $\mathbf{W}$: 가중치 행렬 ($n \times k$)
 - 모든 연결 가중치를 포함
 - 각 행은 각 입력, 각 열은 그 층에 있는 각 뉴런에 해당함
- $\mathbf{b}$: 편향 벡터 ($1 \times k$)
 - 뉴런마다 하나씩 각각 가지고 있는 편향을 모두 포함
 - 행벡터를 행렬에 더하면 행렬의 모든 행에 덧셈이 각각 적용됨(브로드 캐스팅; broadcasting)
- ϕ : 활성화 함수(activation function)
 - 뉴런이 TLU일 경우 계단 함수 사용

$$z_j = w_{1j}x_1 + w_{2j}x_2 + \dots + w_{nj}x_n + b_j$$



w_{ij} : i 번째 입력과 j 번째 출력 뉴런의 연결 가중치
입력: n 차원 벡터가 m 개 ($m \times n$) 행렬
출력: k 차원 벡터가 m 개 ($m \times k$) 행렬

m : 샘플 개수
 n : 입력(특성) 개수
 k : 완전 연결 층 뉴런 개수

한경수

10

퍼셉트론: 학습

- 네트워크가 예측할 때 만드는 오차를 고려하여 학습
- 오차를 감소시키는데 도움이 되는 연결을 강화시킴
 - 한 번에 하나의 샘플이 주입됨
 - 각 샘플에 대해 예측을 만들어 냄
 - 잘못된 예측을 산출하는 모든 출력 뉴런에 대해, 올바른 예측을 만드는데 기여할 법한 입력의 연결 가중치를 강화함

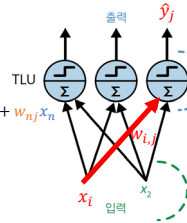
reinforce

- 퍼셉트론 학습 규칙 델타 규칙

$$w_{i,j}^{(next\ step)} = w_{i,j} + \eta(y_j - \hat{y}_j)x_i$$

- $w_{i,j}$: i 번째 입력과 j 번째 출력 뉴런 사이의 연결 가중치
- η : 학습률 오차에 비례하여 각 가중치를 조절함

$y_j = 1, \hat{y}_j = 0$: 오차 > 0 (가중합을 늘려야 함); $x_i > 0 \rightarrow w_{i,j} \uparrow, x_i < 0 \rightarrow w_{i,j} \downarrow$
 $y_j = 0, \hat{y}_j = 1$: 오차 < 0 (가중합을 줄여야 함); $x_i > 0 \rightarrow w_{i,j} \downarrow, x_i < 0 \rightarrow w_{i,j} \uparrow$



11

한경수

퍼셉트론

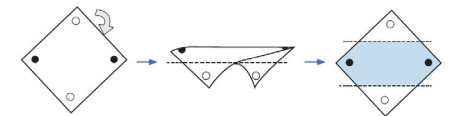
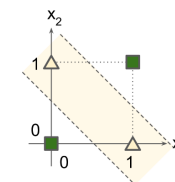
- 퍼셉트론 수렴 정리(perceptron convergence theorem)
 - 학습 샘플이 선형적으로 구분될 수 있다면, 퍼셉트론 학습 알고리즘이 솔루션에 수렴함

- 심각한 약점

- 일부 아주 간단한 문제도 해결하지 못함

- 예: XOR 문제

x_1	x_2	$x_1 \text{ XOR } x_2$
0	0	0
0	1	1
1	0	1
1	1	0



해결책: 평면을 휘어 주어 선 두개를 동시에 긋는 방법
→ 퍼셉트론 두 개를 한 번에 계산 → 은닉층!

출처: 모두의 딥러닝(개정 3판), 조태호 지음, 길벗, 2022, p.116

한경수

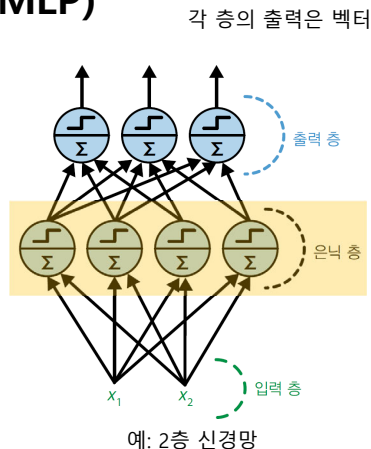
12

다층 퍼셉트론(MLP)

뉴런을 여러 층 쌓아 올리면 일부 제약을 줄일 수 있다는 사실이 밝혀짐

- ◆ 하나의 **입력 층**,
하나 이상의 TLU 층
(**은닉 층**; hidden layer),
하나의 **출력 층**으로 구성

- 입력 층과 가까운 층을
하위 층(lower layer)
출력에 가까운 층을
상위 층(upper layer)라고 함
- 모든 층은 이전 층과
완전 연결(fully connected)



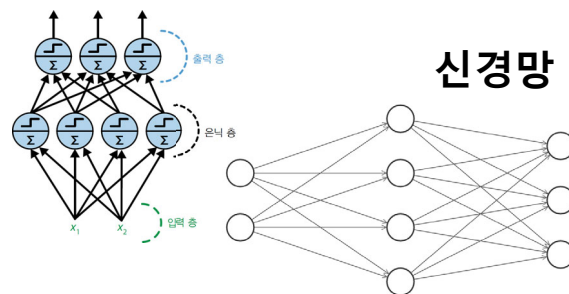
각 층의 출력은 벡터

한경수

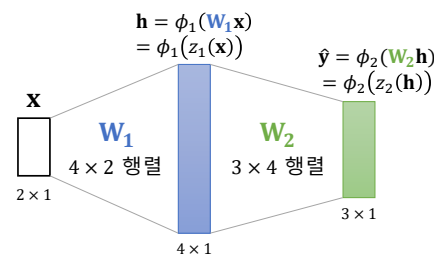
신호가 입력에서 출력 한 방향으로만 흐르는 신경망을 Feedforward NN(FNN)이라 함

13

신경망 표현



2차원 벡터($\mathbf{x}$) $\rightarrow$ 4차원 벡터($\mathbf{h}$) $\rightarrow$ 3차원 벡터($\hat{\mathbf{y}}$)



신경망은 여러 단계의 벡터 변환

신경망의 학습은 적절한 **가중치 값들**을
찾아가는 과정
파라미터
(parameter)

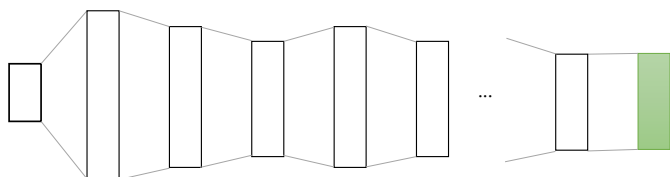
$\hat{\mathbf{y}} = \phi_2 \left(z_2 \left(\phi_1 (z_1(\mathbf{x})) \right) \right)$
신경망은 합성함수!

한경수

14

심층 신경망

- ◆ 심층 신경망(deep neural network; DNN)
 - 은닉층을 여러 개 쌓아 올린 인공 신경망
 - 딥러닝: 심층 신경망을 연구하는 분야



한경수

15

(오차) 역전파 알고리즘

다층 퍼셉트론의 파라미터 학습 방법:
그레이디언트 자동 계산(후진 모드 자동 미분) + 경사 하강법

- Seppo Linnainmaa(1970): 후진 모드 자동 미분 최초 제안 reverse-mode autodiff
- Paul Werbos(1974): 신경망 학습에 이용 제안
- David Rumelhart, [Geoffrey Hinton](#), Ronald Williams(1986): 신경망의 은닉층에 유용한 내부 표현이 학습됨을 실험적으로 입증 $\rightarrow$ 역전파 대중화

Geoffrey Hinton: 2024년 노벨 물리학상 수상

한경수

16

역전파(Backpropagation) 학습 알고리즘

backprop

- 효율적인 기법으로 그래디언트를 자동으로 계산하는 경사 하강법
 - FNN에 적용한 경사 하강법
 - 네트워크를 정방향 한번, 역방향 한번 (총 두 번) 통과하는 것으로 모든 모델 파라미터에 대한 네트워크 오차의 그래디언트 계산
 - 경사 하강법 수행
 - 네트워크가 해결책으로 수렴될 때까지 반복

한경수

17

다층 퍼셉트론

- 알고리즘 수식 유도를 위해 MSE 손실 함수를 사용한다고 가정함

- i 번째 샘플 하나에 대한 손실

$$J(\Theta)^{(i)} = \frac{1}{2} \sum_{k=1}^K (\hat{y}_k^{(i)} - y_k^{(i)})^2$$

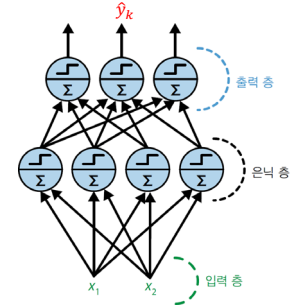
- $\hat{y}_k$: k 번째 출력층 뉴런의 출력
- y_k : k 번째 뉴런의 학습 데이터 정답
- K : 출력층 뉴런 개수

- 전체 샘플에 대한 손실 함수

$$J(\Theta) = \frac{1}{m} \sum_{i=1}^m J(\Theta)^{(i)}$$

$$\frac{\partial J}{\partial w_{11}} = \frac{\partial}{\partial w_{11}} \left(\frac{1}{m} \sum_{i=1}^m J(\mathbf{w}_1, \mathbf{w}_2, \dots)^{(i)} \right) = \frac{1}{m} \sum_{i=1}^m \frac{\partial J^{(i)}}{\partial w_{11}}$$

$$\frac{\partial J}{\partial w_{23}} = \frac{\partial}{\partial w_{23}} \left(\frac{1}{m} \sum_{i=1}^m J(\mathbf{w}_1, \mathbf{w}_2, \dots)^{(i)} \right) = \frac{1}{m} \sum_{i=1}^m \frac{\partial J^{(i)}}{\partial w_{23}}$$

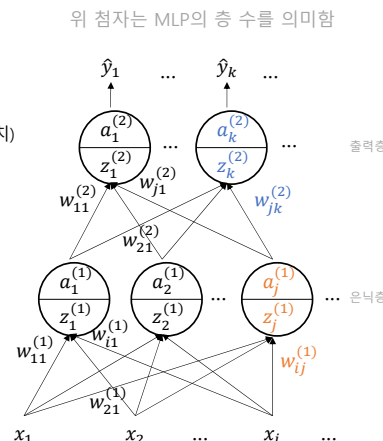


수식 표기의 편의상 $J(\Theta)^{(i)}$, $y_k^{(i)}$, $\hat{y}_k^{(i)}$ 에서 (i) 표기를 생략하여 샘플 하나에 대한 비용 함수의 그래디언트 계산으로 설명함

18

다층 퍼셉트론

- 입력의 i 번째 특성: x_i
- 은닉층(1~(마지막-1)층) j 번째 뉴런의 입력 총합
 - $z_j^{(1)} = \sum_{i=1}^D w_{ij}^{(1)} x_i$
 - $w_{ij}^{(1)}$: i 번째 입력 뉴런과 j 번째 뉴런 사이의 연결 가중치(은닉층 가중치)
 - D : 입력 차원(특성 개수)
- 은닉층 j 번째 뉴런의 출력: $a_j^{(1)} = \phi^{(1)}(z_j^{(1)})$
- 출력층(마지막 층) k 번째 뉴런의 입력 총합
 - $z_k^{(2)} = \sum_{j=1}^M w_{jk}^{(2)} a_j^{(1)}$
 - $w_{jk}^{(2)}$: 은닉층 j 번째 뉴런과 출력층 k 번째 뉴런 사이의 연결 가중치(출력층 가중치)
 - M : 은닉층 뉴런 개수
- 출력층 k 번째 뉴런의 출력
 - $\hat{y}_k = a_k^{(2)} = \phi^{(2)}(z_k^{(2)})$
 - K : 출력 차원(출력층 뉴런 개수)



한경수 $\phi()$: 활성화 함수

19

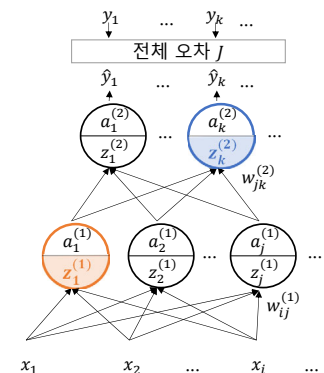
뉴런의 오차

- l 번째 층에 있는 뉴런(유닛) j 의 가중 입력 $z_j^{(l)}$ 이 전체 오차 J 에 미치는 변화율

$$\delta_j^{(l)} = \frac{\partial J}{\partial z_j^{(l)}} \quad (l = 1, 2, \dots)$$

$$\delta_1^{(1)} = \frac{\partial J}{\partial z_1^{(1)}}$$

$$\delta_k^{(2)} = \frac{\partial J}{\partial z_k^{(2)}}$$



한경수

20

$$\delta_k^{(2)} = \frac{2}{K} (a_k^{(2)} - y_k) \phi^{(2)'}(z_k^{(2)})$$

그레디언트 계산

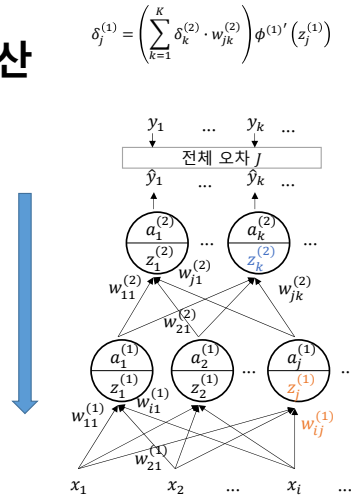
비용함수: MSE

출력층(L) k번째 노드

- $\frac{\partial J}{\partial w_{jk}^{(L)}} = \delta_k^{(L)} \cdot a_j^{(L-1)}$
 - $\delta_k^{(L)} = \frac{2}{K} (a_k^{(L)} - y_k) \phi^{(L)'}(z_k^{(L)})$

은닉층(l) j번째 노드

- $1 \leq l \leq L-1$
- $\frac{\partial J}{\partial w_{ij}^{(l)}} = \delta_j^{(l)} \cdot a_i^{(l-1)}$
 - $\delta_j^{(l)} = \left(\sum_{m=1}^M \delta_m^{(l+1)} \cdot w_{jm}^{(l+1)} \right) \phi^{(l)'}(z_j^{(l)})$
 - M: (l+1)층의 노드 개수
 - $a_i^{(1)} = x_i$
- 은닉층에서의 뉴런 오차($\delta_j^{(l)}$)는 출력층(윗 층)의 뉴런 오차($\delta_m^{(l+1)}$)를 가중치($w_{jm}^{(l+1)}$)에 따라 합하는 형태
 - 출력층의 오차를 반대 방향으로 전달시켜 계산



수식 유도 과정은 참고자료 참조

그레디언트 계산

- 비용 함수: $J(\Theta) = J(\mathbf{W}^{(1)}, \mathbf{W}^{(2)}, \dots, \mathbf{W}^{(L)})$

l층의 그레디언트

$$\mathbf{W}^{(l)}: m \times n$$

- $\nabla_{\mathbf{W}^{(l)}} J(\Theta) = \begin{bmatrix} \frac{\partial}{\partial w_{11}} J(\Theta) & \frac{\partial}{\partial w_{12}} J(\Theta) & \dots & \frac{\partial}{\partial w_{1n}} J(\Theta) \\ \frac{\partial}{\partial w_{21}} J(\Theta) & \frac{\partial}{\partial w_{22}} J(\Theta) & \dots & \frac{\partial}{\partial w_{2n}} J(\Theta) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial w_{m1}} J(\Theta) & \frac{\partial}{\partial w_{m2}} J(\Theta) & \dots & \frac{\partial}{\partial w_{mn}} J(\Theta) \end{bmatrix}$

- n: l층 뉴런 개수, m: (l-1)층 뉴런 개수

$$\frac{\partial J}{\partial w_{jk}^{(L)}} = \delta_k^{(L)} \cdot a_j^{(L-1)}$$

$$\delta_k^{(L)} = \frac{2}{K} (a_k^{(L)} - y_k) \phi^{(L)'}(z_k^{(L)})$$

경사 하강법

출력층: $\mathbf{W}^{(L)(next\ step)} = \mathbf{W}^{(L)} - \eta \nabla_{\mathbf{W}^{(L)}} J(\Theta)$

- $w_{jk}^{(L)(next\ step)} = w_{jk}^{(L)} - \eta \frac{\partial J}{\partial w_{jk}^{(L)}} = w_{jk}^{(L)} - \eta \delta_k^{(L)} a_j^{(L-1)}$

$$= w_{jk}^{(L)} - \eta \frac{2}{K} (a_k^{(L)} - y_k) \phi^{(L)'}(z_k^{(L)}) a_j^{(L-1)}$$

- 출력층의 연결 가중치 업데이트 크기는 은닉층 출력의 크기($a_j^{(L-1)}$)와 출력층에서 생기는 오차($\delta_k^{(L)}$)의 곱으로 결정됨
 - 오차가 없으면 변경되지 않음

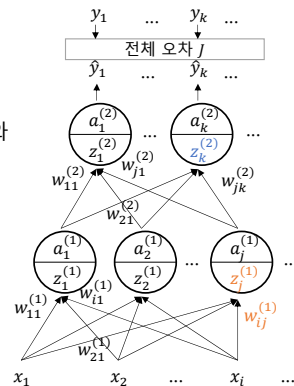
은닉층: $\mathbf{W}^{(l)(next\ step)} = \mathbf{W}^{(l)} - \eta \nabla_{\mathbf{W}^{(l)}} J(\Theta)$

- $1 \leq l \leq L-1$
- $w_{ij}^{(l)(next\ step)} = w_{ij}^{(l)} - \eta \frac{\partial J}{\partial w_{ij}^{(l)}} = w_{ij}^{(l)} - \eta \delta_j^{(l)} a_i^{(l-1)}$

$$= w_{ij}^{(l)} - \eta \left(\sum_{m=1}^M \delta_m^{(l+1)} \cdot w_{jm}^{(l+1)} \right) \phi^{(l)'}(z_j^{(l)}) a_i^{(l-1)}$$

$$\frac{\partial J}{\partial w_{ij}^{(l)}} = \delta_j^{(l)} \cdot a_i^{(l-1)}$$

$$\delta_j^{(l)} = \left(\sum_{m=1}^M \delta_m^{(l+1)} \cdot w_{jm}^{(l+1)} \right) \phi^{(l)'}(z_j^{(l)})$$



$$\delta_k^{(L)} = \frac{2}{K} (a_k^{(L)} - y_k) \phi^{(L)'}(z_k^{(L)})$$

역전파 알고리즘 정리

$$\delta_j^{(l)} = \left(\sum_{m=1}^M \delta_m^{(l+1)} \cdot w_{jm}^{(l+1)} \right) \phi^{(l)'}(z_j^{(l)})$$

연결 가중치 초기화 후,

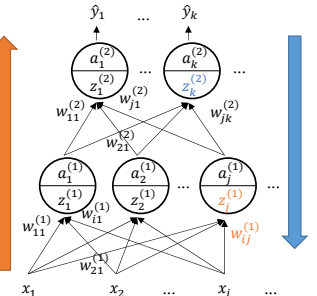
각 미니배치에 대해 전체 샘플을 처리할 때까지 다음 과정 반복

- 정방향 계산
 - 미니배치에 있는 모든 샘플들(x)에 대해, 입력층을 통해 다음 층으로 전달되어 해당 층 모든 뉴런의 출력 계산; 다음 층으로 전달 되어 해당 층 모든 뉴런의 출력 계산; ... 출력 층 모든 뉴런의 출력(y) 계산
 - 중간 계산 값(z, a) 저장

- 역방향 계산(오차 그레디언트 역방향 전파)
 - 출력값과 목표값을 비교하여 출력 층의 오차($\delta^{(L)}$) 계산
 - 은닉층의 연결 가중치가 이 오차에 기여한 정도($\delta^{(l)}$) 계산
 - 입력층에 도달할 때까지 역방향으로 계속됨
- 경사 하강법 단계(연결 가중치 업데이트)
 - 미니배치에 속하는 각 샘플의 그레디언트의 평균 사용

- $\mathbf{W}^{(L)(next\ step)} = \mathbf{W}^{(L)} - \eta \frac{1}{m} \sum_{i=1}^m \nabla_{\mathbf{W}^{(L)}} J(\Theta)^{(i)}$

- $\mathbf{W}^{(l)(next\ step)} = \mathbf{W}^{(l)} - \eta \frac{1}{m} \sum_{i=1}^m \nabla_{\mathbf{W}^{(l)}} J(\Theta)^{(i)}$



연결 가중치

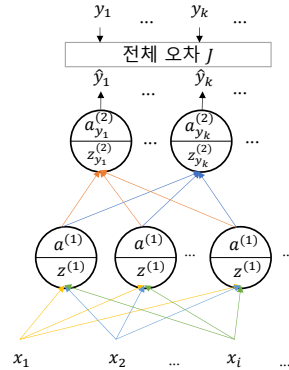
상세 설명은 참고자료 참조

◆ 연결 가중치가 모두 0이거나 같은 값이면,

- 역전파를 통한 학습 후에도 은닉층은 모두 동일한 뉴런
 - 뉴런이 몇 개 있든 하나인 것처럼 작동
 - 정상적인 학습 불가능

◆ 연결 가중치를 랜덤하게 초기화하는 것이 중요함

- 비대칭성
- 전체 뉴런을 다양하게 훈련



한경수

25

활성화 함수

$$\delta_k^{(L)} = \frac{2}{K} (a_k^{(L)} - y_k) \phi^{(L)'}(z_k^{(L)})$$

역전파 알고리즘을 적용하려면 미분 가능한 활성화 함수 필요

◆ 로지스틱(시그모이드) 함수

- $\sigma(z) = \frac{1}{1 + \exp(-z)}$ 0~1
- 0이 아닌 그래디언트가 잘 정의됨

◆ 하이퍼볼릭 탄젠트(쌍곡 탄젠트) 함수

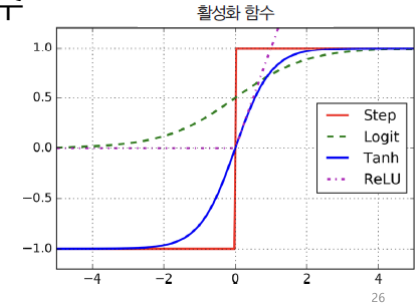
- $\tanh(z) = 2\sigma(2z) - 1$
- 1~1
- 학습 초기에 각 층의 출력이 0을 중심으로 분포하는 경향
 - 양수/음수가 균형을 이루게 됨
 - 수렴 속도 ↑

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

한경수

$$\delta_j^{(l)} = \left(\sum_{m=1}^M \delta_m^{(l+1)} \cdot w_{jm}^{(l+1)} \right) \phi^{(l)'}(z_j^{(l)})$$

역전파 알고리즘 작동을 위해 계단함수를 로지스틱 함수로 변경

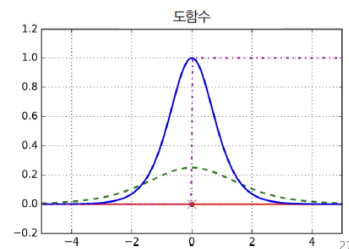
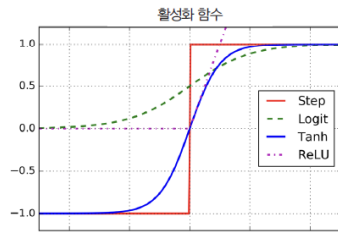


26

활성화 함수

◆ ReLU 함수 rectified linear unit

- $\text{ReLU}(z) = \max(0, z)$
- 계산 속도 빠름
- 출력에 상한 값이 없음
- $z > 0$, 기울기=1
 - 오차 그래디언트를 그대로 역전파시킴
- $z \leq 0$, 기울기=0
 - $z = 0$ 에서 미분 불가능; 기울기를 0으로 간주
 - 오차 그래디언트를 역전파하지 않음



생물학적 뉴런을 구현한 시그모이드 함수보다 ReLU가 인공지능망에서 더 잘 작동함. 생물학적 비유 불필요!

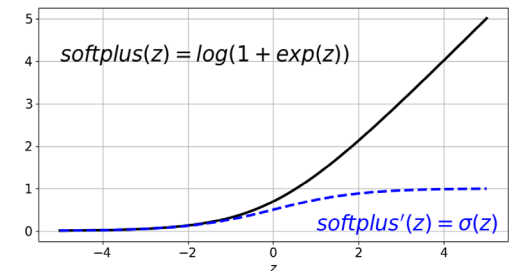
한경수

27

활성화 함수

◆ 소프트플러스(softplus)

- $\text{softplus}(z) = \log(1 + e^z)$
- ReLU의 부드러운 변형
 - $z < 0$: 0에 가깝고; $z > 0$: z 에 가까움
- 도함수: 로지스틱 함수

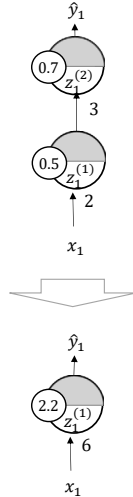


한경수

28

활성화 함수

- ◆ 활성화 함수가 꼭 필요한가?
 - 활성화 함수가 없으면 입력들의 가중합이 그대로 출력됨
 - 뉴런 하나의 역할은 선형 변환
 - $z_1^{(1)} = 2x_1 + 0.5$
 - $z_1^{(2)} = 3z_1^{(1)} + 0.7 = 3(2x_1 + 0.5) + 0.7 = 6x_1 + 2.2$
 - $\hat{y}_1 = 6x_1 + 2.2$
 - 선형 변환을 여러 개 연결하더라도 선형 변환
 - 비선형성을 추가하지 않으면 하나의 층과 마찬가지로



활성화 함수

- ◆ 선형 활성화 함수는 어떤가?
 - 활성화 함수: $\phi(z) = 3z + 0.7$
 - 뉴런의 가중 합: $z = 2x_1 + 0.5$
 - 뉴런의 출력: $a = 3(2x_1 + 0.5) + 0.7 = 6x_1 + 2.2$
 - 비선형 활성화 함수 필요!
- ◆ 비선형 활성화 함수가 있는 충분히 큰 심층 신경망은 이론적으로 어떤 연속 함수도 근사할 수 있음

신경망 구조 및 하이퍼파라미터 설정

회귀를 위한 다층 퍼셉트론(MLP)

- ◆ 은닉 층
 - 층 개수: 문제에 따라 다름
 - 일반적으로 1~5
 - 각 은닉층의 뉴런 수: 문제에 따라 다름
 - 일반적으로 10~100
 - 활성화 함수: ReLU
- ◆ 출력 층
 - 뉴런 수: 예측 차원마다 하나; 예측하는 값의 개수
 - 활성화 함수
 - 출력 값 제한 없으면: 없음; 양수만: ReLU 또는 소프트플러스
 - 특정 범위로 제한: 로지스틱 또는 tanh
- ◆ 손실 함수
 - 전형적으로 MSE, 이상치가 많다면 후버(Huber) 손실

$$MSE(\mathbf{x}, h_{\theta}) = \frac{1}{m} \sum_{i=1}^m (\theta^T \mathbf{x}^{(i)} - y^{(i)})^2$$

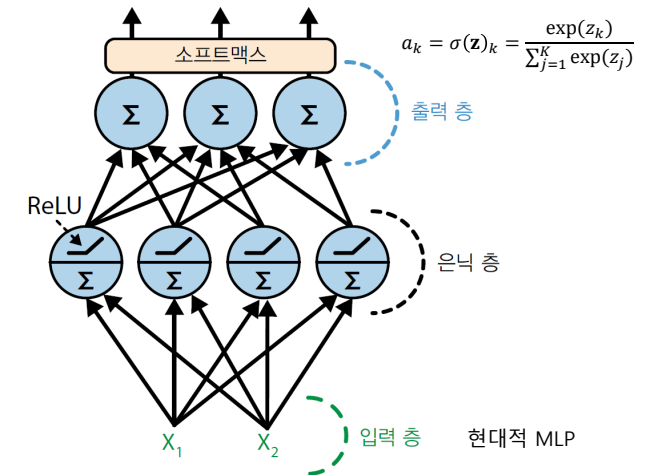
후버 손실: 오차가 임계값 미만이면 이차함수(MSE), 임계값 이상이면 선형 함수(MAE)

분류를 위한 다층 퍼셉트론(MLP)

하이퍼파라미터	이진 분류	다중 레이블 분류	다중 분류
(문제 예)	스팸인지 아닌지	스팸인지 아닌지, 긴급인지 아닌지	숫자 이미지 분류 (클래스가 상호 배타적)
은닉층	회귀와 동일		
출력 뉴런 수	1개	레이블마다 1개	클래스마다 1개
출력층의 활성화 함수	로지스틱 함수	로지스틱 함수	소프트맥스 함수
손실 함수	크로스 엔트로피		

양성 클래스에 대한 예측 확률로 해석

다중 분류를 위한 MLP



신경망 하이퍼파라미터 선택 가이드라인

- ◆ 은닉층 개수
 - 일반 근사 이론: 뉴런 수가 무한하다면 은닉층 하나로 어떤 함수도 근사할 수 있음
 - 심층 신경망이 얇은 신경망보다 파라미터 효율성이 훨씬 좋음
 - 훨씬 적은 수의 뉴런을 사용하여 더 높은 성능 달성
 - 계층적 학습: 아래 층 저수준 구조 모델링(예: 여러 방향의 모양/선) → 중간 층 중간 수준 구조 모델링(예: 사각형, 원) → 상위 층 고수준 구조 모델링(예: 얼굴)
 - 계층 구조는 새로운 데이터에 일반화되는 능력도 향상
 - 얼굴 인식 모델 학습 후, 헤어스타일 인식 신경망을 새로 학습할 경우: 얼굴 인식 모델의 하위 층을 재사용하여 학습 시작 → 전이 학습(transfer learning)
 - 복잡한 문제에는 학습 집합에 과대적합이 발생할 때까지 점진적으로 은닉층 수 증가

신경망 하이퍼파라미터 선택 가이드라인

- ◆ 뉴런 개수
 - 입력층/출력층: 해당 작업에 필요한 입력/출력의 형태에 따라 결정
 - 은닉층:
 - 전통적으로 각 층의 뉴런을 점점 줄여서 구성
 - 최근에는 각 은닉층에 동일 크기 사용; 첫번째 은닉층은 다른 층보다 더 크게 하는 것이 도움이 될 수 있음
 - 필요한 것보다 더 많은 층과 뉴런을 가진 모델 선택한 후, 과대적합되지 않도록 조기 종료나 규제 기법 사용
 - 일반적으로 층의 뉴런 개수보다 층의 개수를 늘리는 쪽이 이득이 많음

신경망 하이퍼파라미터 선택 가이드라인

가장 중요한 하이퍼파라미터: 최적의 학습률 값을 어떻게 알아내는가?

- ◆ 학습률 η
 - 매우 낮은 학습률(예: 10^{-5})에서 시작해서 점진적으로 매우 큰 학습률(예: 10)까지 반복하여 모델 학습
 - 각 학습률에 대한 손실을 파악한 후, 최적의 학습률은 손실이 다시 상승하는 지점보다 조금 아래로 결정
 - 예: 상승점보다 약 10배 낮은 지점
 - 모델 다시 초기화하고 최적 학습률을 사용하여 최종 모델을 학습
 - 최적의 학습률은 다른 하이퍼파라미터(예: 배치 크기)에 의존적
 - 다른 하이퍼파라미터를 수정하면 학습률도 튜닝 필요

신경망 하이퍼파라미터 선택 가이드라인

- ◆ 배치 크기
 - 작은 값(예: 32)이 좋다, 큰 값(예: 8,192)이 좋다 이견
 - 큰 배치 크기를 먼저 시도해 보고,
 - 작은 학습률로 학습을 시작해서 점점 커지도록 하는 학습률 예열(warming up)을 함께 사용
 - 학습이 불안정하거나 최종 성능이 만족스럽지 못하면 작은 배치 크기 사용
- ◆ 활성화 함수
 - 일반적으로 ReLU가 모든 은닉층의 기본값
 - 출력층: 수행하는 작업에 따라 달라짐
- ◆ 학습 반복 횟수
 - 튜닝할 필요 없이 조기 종료 사용

정리



- ◆ 퍼셉트론
- ◆ 다층 퍼셉트론
- ◆ 역전파 알고리즘
 - 가중치 랜덤 초기화
 - 정방향: 모든 층, 모든 뉴런의 출력 계산
 - 역방향: 오차 그레이디언트 역전파
 - 경사 하강법: 연결 가중치 업데이트
- ◆ 신경망 구조 및 하이퍼파라미터 설정
 - 은닉층 개수, 뉴런 개수, 손실 함수
 - 학습률, 배치 크기, 활성화 함수

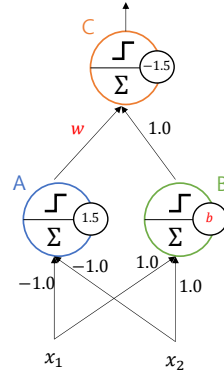
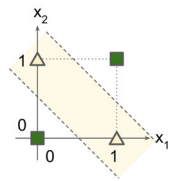


정리 문제 풀이

정리 문제 1: MLP

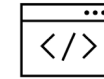
- ◆ 다음과 같은 XOR 문제를 해결하는 다층 퍼셉트론(MLP)을 완성하시오.

x_1	x_2	$x_1 \text{ XOR } x_2$
0	0	0
0	1	1
1	0	1
1	1	0



한경수

41



실습

한경수

42

Keras

- ◆ Tensorflow의 고수준 딥러닝 API
- ◆ Tensorflow 설치시 Keras도 함께 설치됨
 - tf.keras
 - https://www.tensorflow.org/api_docs/python/tf/keras

한경수

43

Keras 기반 학습 진행 절차

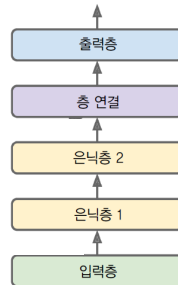
- ◆ 모델 설계
 - 각 층의 연결 구조, 뉴런 개수, 활성화 함수 등 정의
- ◆ 모델 컴파일(compile)
 - 손실 함수(loss), 최적화 방법(optimizer), 평가 척도(metrics) 등 학습 과정 설정
- ◆ 모델 학습(fit)
 - 학습 에포크 횟수(epochs), 미니배치 크기(batch_size) 등을 설정하여 학습 수행
 - 모델의 가중치가 미생성되었으면 학습 시작 전에 build 호출하여 가중치 생성
 - 각 층의 가중치(weights)와 편향(bias) 텐서 생성
 - 입력 데이터의 모양을 알아야 함
- ◆ 모델 평가(evaluate)
- ◆ 모델을 적용하여 새로운 입력에 대해 예측(predict)

한경수

44

모델 설계: Sequential API 사용

- ◆ 모든 층의 입력/출력 연결이 하나씩만 존재하는 가장 단순한 모델로서, 층을 선형으로 쌓음
- ◆ `tf.keras.Sequential`



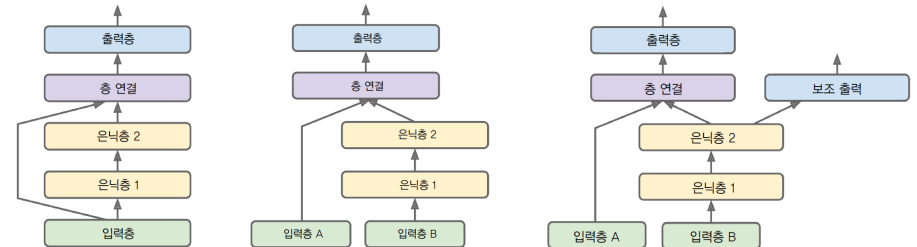
한경수

45

모델 설계: 함수형 API 사용

- ◆ 더 복잡한 모델 생성시 사용
 - 예1: 입력층을 서로 다른 층으로 여러 연결 생성
 - 예2: 입력층의 특성들을 여러 그룹으로 나누어 서로 다른 연결 구조 생성
 - 예3: 연결 구조가 서로 다른 복수의 출력층 생성

은닉층2 = `Dense()`(은닉층1)



한경수

46

모델 설계: 서브클래싱 API 사용

- ◆ `tf.keras.Model` 클래스를 상속하여 동적 모델 생성
 - 생성자 안에 필요한 층 생성
 - `call()` 메소드 안에 수행할 연산 기술
- ◆ 높은 유연성
 - 원하는 다양한 연산들(for, if, ...)을 `call()` 메소드에 포함 가능
- ◆ 제약 사항
 - 모델 구조 파악이나 모델 복제 어려움
 - Keras가 타입/크기를 미리 확인할 수 없음

한경수

47

모델의 활용 및 저장/복원

- ◆ Keras 모델은 일반 층처럼 사용 가능
 - 더 복잡한 모델의 하나의 층으로 사용 가능
- ◆ 모델 저장/복원
 - 저장: `model.save()` 함수
 - 모델 구조, 가중치 등 파라미터들, 옵티마이저 상태 등
 - 복원: `keras.models.load_model()` 함수
 - 로딩된 모델에 학습을 이어갈 수 있음
- ◆ 모델 파라미터만 저장/복원
 - `save_weights()`, `load_weights()` 함수

체크포인트(checkpoint): 모델을 학습하는 동안 특정 시점에 저장된 모델의 상태

한경수

48

실습: 신경망

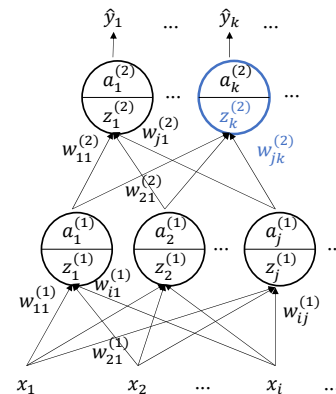
- ◆ 퍼셉트론
 - 단일 TLU 네트워크: `sklearn.linear_model.Perceptron`
- ◆ 다층 퍼셉트론
 - `tensorflow.keras.Sequential`
 - `tensorflow.keras.layers.Dense`



참고자료

그레이디언트 계산: 출력층

- ◆ $w_{jk}^{(2)}$ 를 변수로 놓으면,
 - 다른 파라미터들은 모두 상수로 간주
 - $z_k^{(2)}$ 는 $w_{jk}^{(2)}$ 의 함수
 - $z_k^{(2)} = \sum_{j=1}^M w_{jk}^{(2)} a_j^{(1)}$
 - $z_k^{(2)}$ 가 바뀔때 따라 $a_k^{(2)} = \hat{y}_k$ 도 바뀜
 - $J(\mathbf{w}^{(1)}, \mathbf{w}^{(2)}) = \frac{1}{K} \sum_{k=1}^K (a_k^{(2)} - y_k)^2 = \frac{1}{K} \sum_{k=1}^K (\phi(z_k^{(2)}) - y_k)^2$
 - $w_{jk}^{(2)} \rightarrow z_k^{(2)} \rightarrow a_k^{(2)} \rightarrow J$



$$\begin{aligned} \frac{\partial J}{\partial w_{jk}^{(2)}} &= \frac{\partial J}{\partial z_k^{(2)}} \cdot \frac{\partial z_k^{(2)}}{\partial w_{jk}^{(2)}} = \delta_k^{(2)} \cdot \frac{\partial z_k^{(2)}}{\partial w_{jk}^{(2)}} \\ &= \frac{\partial J}{\partial a_k^{(2)}} \cdot \frac{\partial a_k^{(2)}}{\partial z_k^{(2)}} \cdot \frac{\partial z_k^{(2)}}{\partial w_{jk}^{(2)}} \end{aligned}$$

$$J(\mathbf{w}^{(1)}, \mathbf{w}^{(2)}) = \frac{1}{K} \sum_{k=1}^K (a_k^{(2)} - y_k)^2$$

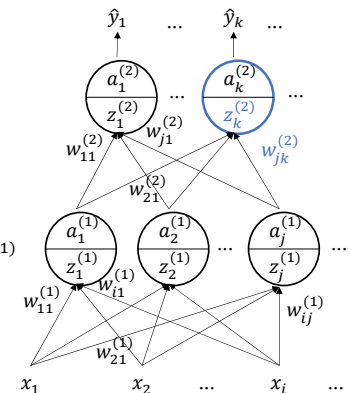
그레이디언트 계산: 출력층

$$a_k^{(2)} = \phi(z_k^{(2)}) \quad z_k^{(2)} = \sum_{j=1}^M w_{jk}^{(2)} a_j^{(1)}$$

$$\begin{aligned} \frac{\partial J}{\partial w_{jk}^{(2)}} &= \delta_k^{(2)} \cdot \frac{\partial z_k^{(2)}}{\partial w_{jk}^{(2)}} = \frac{\partial J}{\partial a_k^{(2)}} \cdot \frac{\partial a_k^{(2)}}{\partial z_k^{(2)}} \cdot \frac{\partial z_k^{(2)}}{\partial w_{jk}^{(2)}} \\ \frac{\partial J}{\partial a_k^{(2)}} &= \frac{\partial}{\partial a_k^{(2)}} \frac{1}{K} \left((a_1^{(2)} - y_1)^2 + \dots + (a_k^{(2)} - y_k)^2 + \dots \right) \\ &= \frac{2}{K} (a_k^{(2)} - y_k) \end{aligned}$$

$$\begin{aligned} \frac{\partial a_k^{(2)}}{\partial z_k^{(2)}} &= \frac{\partial}{\partial z_k^{(2)}} \phi(z_k^{(2)}) = \phi'(z_k^{(2)}) \\ \frac{\partial z_k^{(2)}}{\partial w_{jk}^{(2)}} &= \frac{\partial}{\partial w_{jk}^{(2)}} (w_{1k}^{(2)} a_1^{(1)} + w_{2k}^{(2)} a_2^{(1)} + \dots + w_{jk}^{(2)} a_j^{(1)} + \dots) = a_j^{(1)} \end{aligned}$$

$$\begin{aligned} \frac{\partial J}{\partial w_{jk}^{(2)}} &= \delta_k^{(2)} \cdot a_j^{(1)} \\ \delta_k^{(2)} &= \frac{2}{K} (a_k^{(2)} - y_k) \phi'(z_k^{(2)}) \end{aligned}$$

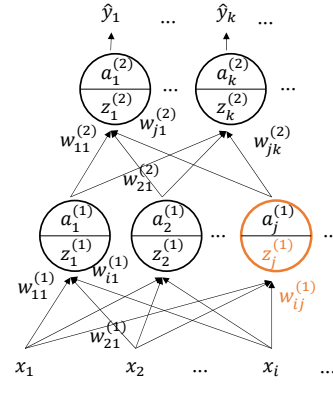


그레이디언트 계산: 은닉층

- $w_{ij}^{(1)}$ 을 변수로 놓으면,
 - 다른 파라미터들은 모두 상수로 간주
 - $z_j^{(1)}$ 는 $w_{ij}^{(1)}$ 의 함수
 - $z_j^{(1)} = \sum_{i=1}^n w_{ij}^{(1)} x_i$
 - $z_j^{(1)}$ 가 바뀔때 따라, $a_j^{(1)}, z_1^{(2)}, \dots, z_k^{(2)}, \dots, a_1^{(2)}, \dots, a_k^{(2)}, \dots$ 도 바뀜
 - $w_{ij}^{(1)} \rightarrow a_j^{(1)} \rightarrow z_1^{(2)}, \dots, z_k^{(2)}, \dots \rightarrow a_1^{(2)}, \dots, a_k^{(2)}, \dots \rightarrow J$
 - J 는 $z_1^{(2)}, \dots, z_k^{(2)}$ 에 대한 다변수 함수

$$\frac{\partial J}{\partial w_{ij}^{(1)}} = \frac{\partial J}{\partial a_j^{(1)}} \cdot \frac{\partial a_j^{(1)}}{\partial z_j^{(1)}} \cdot \frac{\partial z_j^{(1)}}{\partial w_{ij}^{(1)}} = \delta_j^{(1)} \cdot \frac{\partial z_j^{(1)}}{\partial w_{ij}^{(1)}}$$

$$= \frac{\partial J}{\partial a_j^{(1)}} \cdot \frac{\partial a_j^{(1)}}{\partial z_j^{(1)}} \cdot \frac{\partial z_j^{(1)}}{\partial w_{ij}^{(1)}} = \left(\sum_{k=1}^K \frac{\partial J}{\partial z_k^{(2)}} \cdot \frac{\partial z_k^{(2)}}{\partial a_j^{(1)}} \right) \frac{\partial a_j^{(1)}}{\partial z_j^{(1)}} \cdot \frac{\partial z_j^{(1)}}{\partial w_{ij}^{(1)}}$$



하류 그레이디언트 (downstream gradient) ← 상류 그레이디언트 (upstream gradient) × 로컬 그레이디언트 (local gradient)

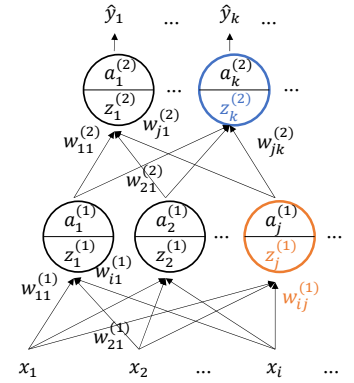
머신러닝 수학 강의자료의 '다변수 합성 함수의 편미분' 부분을 참고할 것

53

그레이디언트 계산: 은닉층

- $\frac{\partial J}{\partial w_{ij}^{(1)}} = \delta_j^{(1)} \cdot \frac{\partial z_j^{(1)}}{\partial w_{ij}^{(1)}} = \left(\sum_{k=1}^K \frac{\partial J}{\partial z_k^{(2)}} \cdot \frac{\partial z_k^{(2)}}{\partial a_j^{(1)}} \right) \frac{\partial a_j^{(1)}}{\partial z_j^{(1)}} \cdot \frac{\partial z_j^{(1)}}{\partial w_{ij}^{(1)}}$
 - $\frac{\partial J}{\partial z_k^{(2)}} = \delta_k^{(2)}$
 - $\frac{\partial z_k^{(2)}}{\partial a_j^{(1)}} = \frac{\partial}{\partial a_j^{(1)}} (w_{1k}^{(2)} a_1^{(1)} + w_{2k}^{(2)} a_2^{(1)} + \dots + w_{jk}^{(2)} a_j^{(1)} + \dots) = w_{jk}^{(2)}$
 - $\frac{\partial a_j^{(1)}}{\partial z_j^{(1)}} = \frac{\partial}{\partial z_j^{(1)}} \phi^{(1)}(z_j^{(1)}) = \phi^{(1)'}(z_j^{(1)})$
 - $\frac{\partial z_j^{(1)}}{\partial w_{ij}^{(1)}} = \frac{\partial}{\partial w_{ij}^{(1)}} (w_{1j}^{(1)} x_1 + w_{2j}^{(1)} x_2 + \dots + w_{ij}^{(1)} x_i + \dots) = x_i$
- $\frac{\partial J}{\partial w_{ij}^{(1)}} = \delta_j^{(1)} \cdot x_i$
 - $\delta_j^{(1)} = \left(\sum_{k=1}^K \delta_k^{(2)} \cdot w_{jk}^{(2)} \right) \phi^{(1)'}(z_j^{(1)})$

예: $\frac{\partial J}{\partial w_{13}^{(1)}} = (\delta_1^{(2)} w_{31}^{(2)} + \delta_2^{(2)} w_{32}^{(2)} + \dots + \delta_K^{(2)} w_{3K}^{(2)}) \cdot \phi^{(1)'}(z_3^{(1)}) \cdot x_1$



그레이디언트 계산 시 뉴런의 오차가 계산되고, 뉴런의 오차는 위 층 모든 뉴런들의 오차에 영향 받음.

54

$$w_{jk}^{(l)(next\ step)} = w_{jk}^{(l)} - \eta \frac{2}{K} (a_k^{(l)} - y_k) \phi^{(l)'}(z_k^{(l)}) a_j^{(l-1)}$$

연결 가중치

- 연결 가중치가 모두 0이라면? $w_{ij}^{(l)} = 0$

정방향1

▪ 각 층마다 모든 뉴런이 동일

- $z_j^{(1)} = 0, a_j^{(1)} = \phi^{(1)}(0) = a^{(1)}$
- $z_k^{(2)} = 0, a_k^{(2)} = \phi^{(2)}(0) = a^{(2)}$

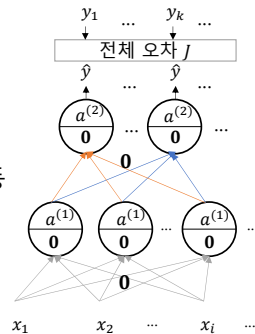
역방향/업데이트1

- 목표값이 동일한 출력층 노드는 은닉층과 관계 없이 동일한 가중치를 가짐

$$\text{➢ } w_{jk}^{(2)(next\ step)} = 0 - \eta \frac{2}{K} (a_k^{(2)} - y_k) \phi^{(2)'}(0) a^{(1)} = w_{jk}^{(2)(next\ step)}$$

- 은닉층의 모든 노드의 가중치는 0으로 변함 없음

$$\text{➢ } w_{ij}^{(1)(next\ step)} = 0 - \eta \left(\sum_{k=1}^K \delta_k^{(2)} \cdot 0 \right) \phi^{(1)'}(0) x_i = 0$$



한경수

55

$$w_{jk}^{(l)(next\ step)} = w_{jk}^{(l)} - \eta \frac{2}{K} (a_k^{(l)} - y_k) \phi^{(l)'}(z_k^{(l)}) a_j^{(l-1)}$$

연결 가중치

- 연결 가중치가 모두 0이라면?(계속) $w_{ij}^{(l)} = 0$

정방향2

▪ 은닉층: 역시 모든 뉴런이 동일

- $z_j^{(1)} = 0, a_j^{(1)} = \phi^{(1)}(0) = a^{(1)}$
- 출력층: 목표값이 같은 뉴런은 출력값도 동일
- $z_k^{(2)} = \sum_{j=1}^M w_{jk}^{(2)} a_j^{(1)} = z_{yk}^{(2)}, a_k^{(2)} = \phi^{(2)}(z_k^{(2)}) = a_{yk}^{(2)}$

역방향/업데이트2

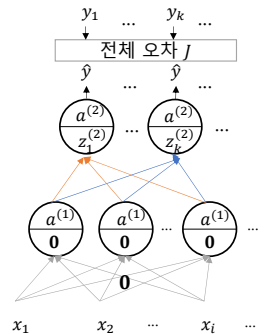
- 목표값이 동일한 출력층 노드는 은닉층과 관계 없이 동일한 가중치를 가짐

$$\text{➢ } w_{jk}^{(2)(next\ step)} = w_{yk}^{(2)} - \eta \frac{2}{K} (a_k^{(2)} - y_k) \phi^{(2)'}(z_{yk}^{(2)}) a^{(1)} = w_{yk}^{(2)(next\ step)}$$

- 은닉층의 모든 노드의 가중치는 여전히 0

$$\text{➢ } w_{ij}^{(1)(next\ step)} = 0 - \eta \left(\sum_{k=1}^K \delta_k^{(2)} \cdot 0 \right) \phi^{(1)'}(z^{(1)}) x_i = 0$$

➢ 은닉층의 가중치는 업데이트되지 않음. 학습되지 않음.



한경수

56

$$w_{jk}^{(l)(next\ step)} = w_{jk}^{(l)} - \eta \frac{2}{K} (a_k^{(l)} - y_k) \phi^{(l)'}(z_k^{(l)}) a_j^{(l-1)}$$

연결 가중치

- ◆ 연결 가중치가 모두 같은 값이라면?

$$w_{ij}^{(l)} = w^{(l)}$$

- 정방향1

- 각 층마다 모든 뉴런이 동일

$$\triangleright z_j^{(1)} = \sum_{i=1}^D w^{(1)} x_i = z^{(1)}, a_j^{(1)} = \phi^{(1)}(z^{(1)}) = a^{(1)}$$

$$\triangleright z_k^{(2)} = \sum_{j=1}^M w^{(2)} a^{(1)} = z^{(2)}, a_k^{(2)} = \phi^{(2)}(z^{(2)}) = a^{(2)}$$

- 역방향/업데이트1

- 목표값이 동일한 출력층 노드는 은닉층과 관계 없이 동일한 가중치를 가짐

$$\triangleright w_{jk}^{(2)(next\ step)} = w^{(2)} - \eta \frac{2}{K} (a^{(2)} - y_k) \phi^{(2)'}(z^{(2)}) a^{(1)} = w_{jk}^{(2)(next\ step)}$$

- 입력값이 동일한 은닉층 노드는 동일한 가중치를 가짐

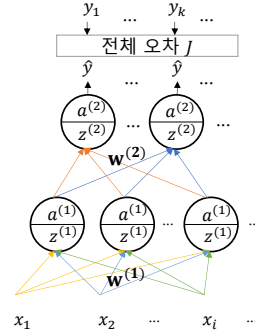
$$\triangleright w_{ij}^{(1)(next\ step)} = w^{(1)} - \eta \left(\sum_{k=1}^K \delta_k^{(2)} \cdot w_{jk}^{(2)} \right) \phi^{(1)'}(z^{(1)}) x_i = w_{ij}^{(1)(next\ step)}$$

- 은닉층이 여러 개인 경우, 중간에 끼인 은닉층의 모든 노드의 가중치가 같은 값으로 업데이트됨

$$w_{ij}^{(l)(next\ step)} = w^{(l)} - \eta \left(\sum_{m=1}^M \delta_m^{(l+1)} \cdot w_{jm}^{(l+1)} \right) \phi^{(l)'}(z^{(l)}) a_i^{(l-1)} = w_{ij}^{(l)(next\ step)}$$

한경수

57



$$w_{jk}^{(l)(next\ step)} = w_{jk}^{(l)} - \eta \frac{2}{K} (a_k^{(l)} - y_k) \phi^{(l)'}(z_k^{(l)}) a_j^{(l-1)}$$

연결 가중치

- ◆ 연결 가중치가 모두 같은 값이라면?(계속)

$$w_{ij}^{(l)} = w^{(l)}$$

- 정방향2

- 은닉층: 역시 모든 뉴런이 동일

$$\triangleright z_j^{(1)} = \sum_{i=1}^D w_i^{(1)} x_i = z^{(1)}, a_j^{(1)} = \phi^{(1)}(z^{(1)}) = a^{(1)}$$

- 출력층: 목표값이 같은 뉴런은 출력값도 동일

$$\triangleright z_k^{(2)} = \sum_{j=1}^M w_{jk}^{(2)} a^{(1)} = z^{(2)}, a_k^{(2)} = \phi^{(2)}(z^{(2)}) = a^{(2)}$$

- 역방향/업데이트2

- 목표값이 동일한 출력층 노드는 은닉층과 관계 없이 동일한 가중치를 가짐

$$\triangleright w_{jk}^{(2)(next\ step)} = w_{jk}^{(2)} - \eta \frac{2}{K} (a^{(2)} - y_k) \phi^{(2)'}(z^{(2)}) a^{(1)} = w_{jk}^{(2)(next\ step)}$$

- 입력값이 동일한 은닉층 노드는 동일한 가중치를 가짐

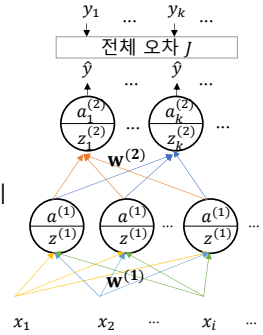
$$\triangleright w_{ij}^{(1)(next\ step)} = w_i^{(1)} - \eta \left(\sum_{k=1}^K \delta_k^{(2)} \cdot w_{jk}^{(2)} \right) \phi^{(1)'}(z^{(1)}) x_i = w_{ij}^{(1)(next\ step)}$$

- 은닉층이 여러 개인 경우, 중간에 끼인 은닉층의 모든 노드의 가중치가 같은 값으로 업데이트됨

$$w_{ij}^{(l)(next\ step)} = w^{(l)} - \eta \left(\sum_{m=1}^M \delta_m^{(l+1)} \cdot w_{jm}^{(l+1)} \right) \phi^{(l)'}(z^{(l)}) a_i^{(l-1)} = w_{ij}^{(l)(next\ step)}$$

한경수

58



층마다 모든 뉴런이 동일하게 유지되어, 좋은 솔루션으로 수렴하지 못함

$$w_{jk}^{(l)(next\ step)} = w_{jk}^{(l)} - \eta \frac{2}{K} (a_k^{(l)} - y_k) \phi^{(l)'}(z_k^{(l)}) a_j^{(l-1)}$$

연결 가중치

- ◆ 연결 가중치가 모두 같은 값이라면?

$$w_{ij}^{(l)} = w$$

- 정방향1

- 각 층마다 모든 뉴런이 동일

$$\triangleright z_j^{(1)} = \sum_{i=1}^D w^{(1)} x_i = z^{(1)}, a_j^{(1)} = \phi^{(1)}(0) = a^{(1)}$$

$$\triangleright z_k^{(2)} = 0, a_k^{(2)} = \phi^{(2)}(0) = a^{(2)}$$

- 역방향/업데이트1

- 목표값이 동일한 출력층 노드는 은닉층과 관계 없이 동일한 가중치를 가짐

$$\triangleright w_{jk}^{(2)(next\ step)} = 0 - \eta \frac{2}{K} (a^{(2)} - y_k) \phi^{(2)'}(0) a^{(1)} = w_{jk}^{(2)(next\ step)}$$

- 입력값이 동일한 은닉층 노드는 동일한 가중치를 가짐

$$\triangleright w_{ij}^{(1)(next\ step)} = 0 - \eta \left(\sum_{k=1}^K \delta_k^{(2)} \cdot w_{jk}^{(2)} \right) \phi^{(1)'}(0) x_i = w_{ij}^{(1)(next\ step)}$$

- 중간에 끼인 은닉층이면, 모든 노드의 모든 가중치가 같은 값으로 업데이트됨

$$w_{ij}^{(l)(next\ step)} = 0 - \eta \left(\sum_{m=1}^M \delta_m^{(l+1)} \cdot w_{jm}^{(l+1)} \right) \phi^{(l)'}(z_j^{(l)}) a_i^{(l-1)}$$

한경수

59

